

Ensemble-learning error mitigation for variational quantum shallow-circuit classifiers

Qingyu Li¹, Yuhan Huang², Xiaokai Hou¹, Ying Li^{3,*}, Xiaoting Wang^{1,4,†} and Abolfazl Bayat^{1,4,‡}¹*Institute of Fundamental and Frontier Sciences, University of Electronic Science and Technology of China, Chengdu, 611731, China*²*The Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, 999077, Hong Kong, China*³*Graduate School of China Academy of Engineering Physics, Beijing 100193, China*⁴*Key Laboratory of Quantum Physics and Photonic Quantum Information, Ministry of Education, University of Electronic Science and Technology of China, Chengdu 611731, China*

(Received 26 February 2023; revised 30 October 2023; accepted 11 December 2023; published 10 January 2024)

Classification is one of the main applications of supervised learning. Recent advancements in developing quantum computers have opened a new possibility for machine learning on such machines. Due to the noisy performance of near-term quantum computers, error mitigation techniques are essential for extracting meaningful data from noisy raw experimental measurements. Here, we propose two ensemble-learning error mitigation methods, namely, bootstrap aggregating and adaptive boosting, which can significantly enhance the performance of variational quantum classifiers for both classical and quantum datasets. The idea is to combine several weak classifiers, each implemented on a shallow noisy quantum circuit, to make a strong one with high accuracy. While both of our protocols substantially outperform error-mitigated primitive classifiers, the adaptive boosting shows better performance than the bootstrap aggregating. The protocols have been exemplified for classical handwriting digits as well as quantum phase discrimination of a symmetry-protected topological Hamiltonian, in which we observe a significant improvement in accuracy. Our ensemble-learning methods provide a systematic way of utilizing shallow circuits to solve complex classification problems.

DOI: [10.1103/PhysRevResearch.6.013027](https://doi.org/10.1103/PhysRevResearch.6.013027)

I. INTRODUCTION

Machine learning, as a method in which computers learn patterns within data, has revolutionized almost all aspects of our lives [1]. Classification algorithms are among the most important types of machine learning tasks with a wide range of applications in finance, business, industry, marketing, and scientific research [2,3]. In these algorithms, all data are divided into a few discrete classes that contain elements with certain common features. So far, numerous classification algorithms have been developed, such as logistic regression [4], decision trees [5], k -nearest neighbors [6], support vector machines [7], and neural network classifiers [8]. The sophistication that big data brings to the training process may decrease the accuracy of algorithms. To overcome this, one can adopt ensemble-learning methods in which several classifiers are combined to make a stronger one with higher prediction accuracy. The most prominent ensemble-learning methods are bootstrap aggregating (Bagging) [9] and adaptive boosting (AdaBoost) [10,11] which have been developed in the classical machine learning context.

Quantum computers are rapidly emerging in various physical platforms, including superconducting qubits [12–17], ion traps [18–22], optical lattices [23–25], Rydberg atoms [26], and photonic chips [27–29]. They can push our computational power well beyond the capability of existing classical computers [14,16,28,30]. Indeed, several classification algorithms have been generalized to be adopted on quantum computers, including distance-based quantum classifier [31], quantum support vector machine [32,33], quantum k -nearest-neighbor algorithm [34,35], quantum decision tree classifiers [36,37], and quantum neural networks [38–47]. However, current noisy intermediate-scale quantum (NISQ) computers are far away from achieving fault-tolerant quantum computing [48] and thus most of these algorithms cannot be realized on such noisy hardware. In fact, outperforming classical computers with NISQ devices is still a hotly debated subject [48].

Variational quantum algorithms (VQAs) [49] are the most promising approach for achieving quantum advantage on NISQ computers. In these algorithms, the complexity is divided between a quantum circuit and a classical computer, allowing a complex task to be achieved using a shallow quantum circuit. So far, VQAs have been exploited to solve a wide range of problems, including eigenvalue solvers [50–55], quantum neural networks [41,56–58], quantum adversarial machine learning [59–62], quantum approximate optimization algorithms [63], linear equation solvers [64–66], and quantum sensing [67–69]. Variational Quantum Classification (VQC) algorithms, as typical VQAs, have also been developed to solve classification problems on NISQ computers [39,40,44,60,62,70–73], with some of them being

*yli@gascaep.ac.cn

†xiaoting@uestc.edu.cn

‡abolfazl.bayat@uestc.edu.cn

experimentally demonstrated [14,62,74,75]. Nonetheless, the imperfect nature of NISQ computers restricts the achievable accuracy of VQCs. In the absence of error correction, error mitigation techniques [76–81] have been developed to extract noise-free values from noisy experimental data. Zero-noise extrapolation (ZNE) is one of the most practical error mitigation methods, in which one can measure the desired observable at different levels of noise. Then by extrapolation, one can estimate the noise-free value through a proper fitting function [76–78]. Developing novel error-mitigation techniques is essential for achieving practical quantum advantage using near-term quantum computers. Indeed, one open problem is whether one can combine successful ensemble learning techniques, developed in classical machine learning, with VQCs to enhance the precision of classifiers for both classical and quantum datasets.

In this paper, we develop two ensemble-learning error mitigation techniques, namely, Bagging and AdaBoost, for VQCs to combine a few weak quantum classifiers and make a strong one with enhanced accuracy. This allows to use of shallow circuits for each of the classifiers and improves noise resilience. We learn the trainability and the generalization with various training sample sizes of our ensemble-learning VQCs in a noise-free environment. The results show that AdaBoost VQC has better trainability, namely, fewer training iterations, and better generalization, namely, a smaller generalization gap and higher test accuracy, than those VQCs without ensemble-learning error mitigation techniques. When we consider the decoherence noise in the quantum computer, We find that both proposed protocols significantly outperform the ZNE method in classification tasks. In the two protocols, the AdaBoost shows stronger performance, namely, higher accuracy and more noise resilience, than the Bagging. Since our ensemble learning algorithms can achieve accurate classification with only shallow circuits, they are NISQ-friendly and feasible for applications using existing quantum-computing technologies. This makes them very distinct from other quantum versions of ensemble-learning algorithms [82–86] which utilize deep-circuit quantum subroutines, such as quantum phase estimation [87], quantum means estimation [88,89] and Grover search [90] algorithms, to speed up the training process and reduce the sample complexity.

We noted that a plain ensemble learning method, namely, plurality voting, had been applied to improve the performance of VQCs [91]. However, the plurality voting method is mainly used to reduce variance and improve the robustness of the model to data, but can not significantly improve the accuracy of the model.

II. VARIATIONAL QUANTUM CLASSIFIERS

A. Classification problems

Classification tasks are types of supervised machine learning problems in which the goal is to predict a discrete class label y , for a given unknown data $\mathbf{x}$. In general, the classifier is trained by a labeled training dataset with M_{tra} samples, $\hat{\mathcal{D}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{M_{\text{tra}}}$, where $\mathbf{x}_i = [x_{i1}, \dots, x_{iN_f}]^T$ represents an input vector with N_f features and y_i is the corresponding class label which takes K_c different values (i.e., $y_i \in \{1, \dots, K_c\}$). After

training, the classifier can be described as a map $\hat{y} = f(\mathbf{x})$ where $\hat{y}$ is the predicted label for a given input $\mathbf{x}$. For a good classifier, we expect that $y = \hat{y}$, namely, predicting the correct class label. In reality, our prediction might be wrong for some inputs, nonetheless, the objective is to keep the ratio of wrong predictions as small as possible.

Recent advancements in developing quantum computers have opened a new territory for exploiting such machines for solving classification problems. In this case, apart from solving conventional classification problems, which deal with classical datasets, one can also consider quantum datasets $\hat{\mathcal{D}} = \{(|\mathbf{x}_i\rangle, y_i)\}_{i=1}^{M_{\text{tra}}}$, where $|\mathbf{x}_i\rangle$ is a quantum state to represent the input features and y_i is the corresponding class label which takes K_c different values. The inherent nature of quantum datasets $\hat{\mathcal{D}}$ justifies the use of a quantum classifier as no classical counterpart can be used for such data. The situation is, however, very different for classical datasets as it is still an open question whether the full capacity of quantum computers can be exploited for the classification of classical data.

In general, one constructs a classifier $f(\mathbf{x})$ by training it on dataset $\hat{\mathcal{D}}$, which can be either classical or quantum. We denote the accuracy of $f(\mathbf{x})$ as $1-e$, where e is error rate as

$$e = \frac{1}{M_{\text{tra}}} \sum_{i=1}^{M_{\text{tra}}} \mathbb{I}(f(\mathbf{x}_i) \neq y_i), \quad (1)$$

where $\mathbb{I}(\cdot)$ is the indicator function with $\mathbb{I}(\cdot) = 1$ for $(\cdot)$ being true and $\mathbb{I}(\cdot) = 0$ otherwise. A random guess determines the class label correctly with a probability $1/K_c$ and thus its error rate, given in Eq. (1), would be $e = (K_c - 1)/K_c$. A given classifier $f(\mathbf{x})$ if called strong if $e \sim 0$ and is called weak if $e \sim (K_c - 1)/K_c$.

B. General implementation of variational quantum classifiers

Variational quantum algorithms (VQAs) are the most promising approach for achieving quantum advantage on NISQ computers. In these algorithms, the complexity is divided between a quantum circuit and a classical optimizer. Therefore even a shallow quantum circuit might be sufficient to achieve a complex task. Recently, VQAs have also been used for developing quantum classifiers for both classical [39,40,44] and quantum [40,41,73] datasets. Nonetheless, in the NISQ era, developing new techniques for mitigating the effect of noise is essential for scaling up the classification algorithms to deal with more complex datasets which normally demand larger numbers of qubits and deeper circuit depths.

In this work, we focus on variational quantum classifiers (VQC). The VQC circuits contain three parts: encoding circuit, parameterized circuit, and measurement. Without loss of generality, the schematic representation of the circuit is shown in Fig. 1(a) (noted that other ingenious quantum circuits [92] are also alternative.) For quantum datasets, the encoding circuit is not needed as the data can directly be fed into the parameterized circuit. For classical datasets, however, the input data $\mathbf{x}_i$ has to be encoded into a quantum state $|\mathbf{x}_i\rangle$. Amplitude encoding is the most efficient way for converting classical data into a quantum state with an exponential advantage through

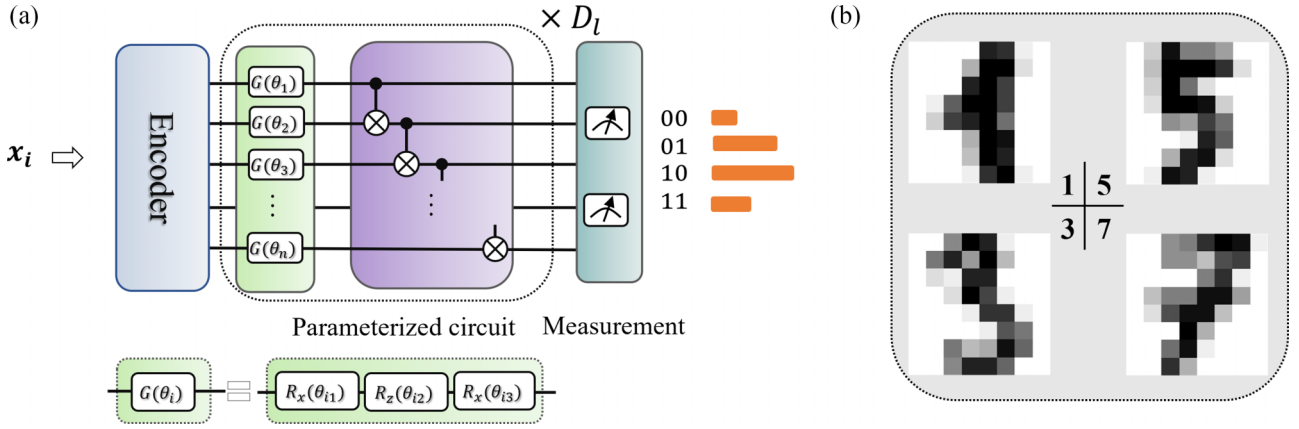


FIG. 1. Circuit design and the classical dataset. (a) The quantum circuit used in our ensemble-learning VQC protocols contains three different parts, namely, encoder, parameterized circuit, and measurement. The encoder part transforms classical input data into a quantum state. While in the paper, we use amplitude encoding the protocol works equally well for rotation encoding too. For quantum datasets, the encoder part is not needed. The prepared quantum states are fed into a parameterized circuit in which each qubit first undergoes a local rotation $G^{(q)}(\theta_q) = R_x^{(q)}(\theta_{q1})R_z^{(q)}(\theta_{q2})R_x^{(q)}(\theta_{q3})$, shown in the lower panel, and then a series of two-qubit controlled-not gates act on nearest-neighbor qubits. The whole parameterized circuit is repeated D_l times. Then the quantum measurement is applied to a few qubits which depend on the number of classes, for obtaining the probabilities of different labels. The label with the largest probability is chosen as the final prediction label. (b) Four typical images of the MNIST dataset that show handwriting digits 1, 3, 5, 7. Each image contains 8×8 pixels which are reshaped as a normalized 64-dimensional vector $\mathbf{x}_i$ as the input data. The dataset has 1541 training samples and 726 test samples with these four digits.

mapping N_f features into $N_q = \lceil \ln(N_f) \rceil$ qubits as

$$\mathbf{x}_i \rightarrow |\mathbf{x}_i\rangle = \frac{1}{\|\mathbf{x}_i\|} \sum_{j=1}^{N_f} x_{ij} |j\rangle, \quad (2)$$

where $\|\mathbf{x}_i\| = \sqrt{\mathbf{x}_i^T \mathbf{x}_i}$ is the norm of $\mathbf{x}_i$ and $|j\rangle$ is a quantum state of N_q qubits with binary representation of j in the computational basis. This encoding is assumed to be done through a quantum random access memory (QRAM) module [93–96]. It is worth emphasizing that our protocol does not depend on any specific encoding method and can easily be generalized to other encoders, such as rotation encoding [57,97,98]. Therefore, for the sake of brevity, we only focus on amplitude encoding. The output of the encoder is fed into a parameterized circuit that contains several layers. Each layer of the parameterized circuit starts with a series of local rotations $\prod_q G^{(q)}(\theta_q)$ acting on all qubits with

$$G^{(q)}(\theta_q) = R_x^{(q)}(\theta_{q1})R_z^{(q)}(\theta_{q2})R_x^{(q)}(\theta_{q3}), \quad (3)$$

where $\theta_q = [\theta_{q1}, \theta_{q2}, \theta_{q3}]^T$, $R_\alpha^{(q)}(\theta) = e^{-i\theta\sigma_\alpha^{(q)}/2}$ (for $\alpha = x$ or z) and $\sigma_\alpha^{(q)}$ is the Pauli operator α acting on qubit q . The single qubit rotations are followed by a series of two-qubit controlled-not gates $\prod_q U_{CX}^{(q,q+1)}$ with

$$U_{CX}^{(q,q+1)} = |0\rangle\langle 0|^{(q)} \otimes I^{(q+1)} + |1\rangle\langle 1|^{(q)} \otimes \sigma_x^{(q+1)}, \quad (4)$$

where $I^{(q)}$ represents identity acting on qubit q . Therefore the action of the parameterized circuit with D_l layers on N_q qubits can be described by a unitary operator of the form

$$U(\theta) = \prod_{d=1}^{D_l} \left(\prod_{q=1}^{N_q-1} U_{CX}^{(q,q+1)} \prod_{q=1}^{N_q} G^{(q)}(\theta_{dq}) \right). \quad (5)$$

The schematic of the circuit is shown in Fig. 1(a). The output of the circuit is given by $U(\theta)|\mathbf{x}_i\rangle$. By measuring the last few qubits of the circuit one can determine the class label of the input $|\mathbf{x}_i\rangle$. In fact, the number of qubits that are measured is determined by $\lceil \ln(K_c) \rceil$. The measurement outcomes can be described by projectors $\{\Pi_k\}^{K_c}$, where K_c is the number of classes. For instance, for binary classification (i.e., $K_c = 2$), only the last qubit is measured and the projectors are given by $\Pi_0 = |0\rangle\langle 0|$ and $\Pi_1 = |1\rangle\langle 1|$, acting on qubit N_q . Similarly, for a four-class problem, one has to measure the last two qubits (namely, qubits N_q-1 and N_q), and the classes are determined by projectors $\Pi_k \in \{|00\rangle\langle 00|, |01\rangle\langle 01|, |10\rangle\langle 10|, |11\rangle\langle 11|\}$, which act on qubits N_q-1 and N_q . The probabilities of measurement outcomes are considered as the probabilities of obtaining each class label $\hat{y}_i$, as $\mathbf{p}_i(\mathbf{x}_i, \theta) = [p_{i1}, \dots, p_{iK_c}]^T$, where

$$p_{ik}(\mathbf{x}_i, \theta) = \langle \mathbf{x}_i | U^\dagger(\theta) \Pi_k U(\theta) | \mathbf{x}_i \rangle. \quad (6)$$

The label $\hat{y}_i$ is determined by the class k whose probability p_{ik} is maximum, namely, $\hat{y}_i = \arg \max_k p_{ik}$. The schematic of the procedure is shown in Fig. 1(a). The performance of VQC is evaluated by a loss function described by cross-entropy

$$\mathcal{L}(\hat{\mathcal{D}}; \theta) = -\frac{1}{M_{\text{tra}}} \sum_{i=1}^{M_{\text{tra}}} \mathbf{y}_i^T \ln(\mathbf{p}_i(\mathbf{x}_i, \theta)), \quad (7)$$

where $\mathbf{y}_i = [y_{i1}, \dots, y_{iK_c}]^T$ is the one-hot encoding of the true class label y_i with only one of the elements y_{ik} , which is the right class label, is 1 and the rest are 0. By using Adam optimizer [99], which is a gradient-based method, one can iteratively update θ in order to minimize the loss function. More details about the training can be found in the Appendix section. For an optimal θ^* where the loss function converges to its minimum the quantum circuit is trained and can be used

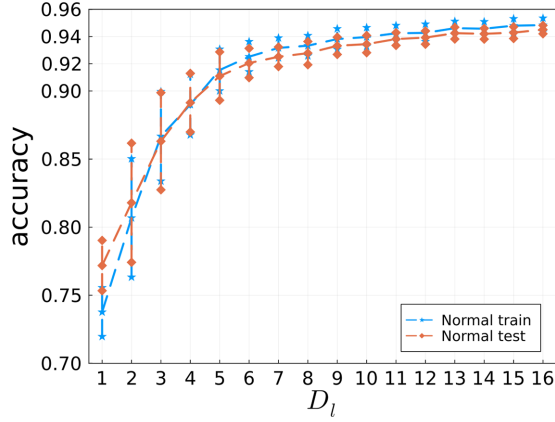


FIG. 2. Increasing layers in noise-free quantum circuits. The training and testing accuracies of normal VQC for classifying MNIST dataset with odd digits $\{1, 3, 5, 7\}$ are shown as a function of the depth D_l of the parameterized circuit. Each of the data points is averaged over 50 different random initial sets of parameters and the error bars represent their standard deviation. The performance monotonically improves by increasing the circuit layers in a noise-free quantum computer. The closeness of the two types of accuracies shows that the training does not impose over-fitting.

for classifying unseen data $\mathbf{x}$

$$\hat{y} = f(\mathbf{x}; \boldsymbol{\theta}^*) = \arg \max_k \langle \mathbf{x} | U^\dagger(\boldsymbol{\theta}^*) \Pi_k U(\boldsymbol{\theta}^*) | \mathbf{x} \rangle. \quad (8)$$

Our classifier is considered a strong one if $\hat{y} = f(\mathbf{x}; \boldsymbol{\theta}^*)$ assigns the correct class label to most of the unseen data $\mathbf{x}$.

C. Performance of VQC for classical datasets

In order to show the performance of VQC for classifying classical data, we consider the MNIST dataset which contains handwriting digital images with 8×8 pixels (i.e., $N_f = 64$ features) [100]. Each pixel takes a number between 0 (perfectly white) to 1 (perfectly black). For the sake of simplicity and without loss of generality we only consider odd numbers and thus our classification has four different classes, labeled by digits 1, 3, 5, and 7. A typical image for each of these four classes is presented in Fig. 1(b). The dataset contains 2267 samples from which 1541 samples are used for training and the 726 unseen samples are used for testing the accuracy. We train the circuit shown in Fig. 1(a) with $N_q = 6$ for various layers D_l . In Fig. 2, we plot both the training and test accuracies as a function of circuit layers D_l . Each data point is averaged over 50 random initialization of the circuit parameters. The error bars show the variation of accuracy across these 50 repetitions. Furthermore, both training and test accuracies are very close to each other which shows that the training is not affected by overfitting. Due to this, in what follows, we only report test accuracy as a quantification measure for the quality of our procedure. In addition, the accuracy is improved rapidly up to $D_l \sim 7$ layers before entering a slow convergence regime. In fact, one needs $D_l = 12$ layers to achieve an accuracy of 0.94, and even for up to $D_l = 16$ one cannot still reach an accuracy of 0.95. By increasing the number of layers the error bars decrease

indicating robustness against parameter initialization. Note that our quantum circuit is noise-free and all quantum gates operate perfectly. That is why the accuracy keeps improving by increasing the layers. In practice, since gates are imperfect and each of them induces noise in the system the accuracy has a more complex dependence on the circuit depth as will be discussed in the following sections.

D. VQC with ZNE

NISQ quantum computers suffer from gate operations and short qubit coherence times. While single-qubit operations can be achieved with fidelity ~ 0.999 [101], the two-qubit gates are more susceptible to noise. For the sake of simplicity, in order to simulate the effect of noise in NISQ computers one can consider two-qubit gates as the only source of noise in the system. In this paper, we emulate the effect of noise as a depolarizing channel which affects the operation of controlled-not gates on qubits q and $q + 1$ as

$$\xi_{\text{CX}}(\rho^{(q,q+1)}) = \frac{P}{4} I^{(q,q+1)} + (1-P) U_{\text{CX}}^{(q,q+1)} \rho U_{\text{CX}}^{(q,q+1)\dagger}, \quad (9)$$

where P quantifies the strength of decoherence. Note that this noise model is very pessimistic as the output is considered to be a maximally mixed state with probability P which means that decoherence kills all the information in the system. In order to reduce the impact of noise in near-term quantum computers, error mitigation techniques [76–81] have been developed for post-processing the noisy data. In this paper, we use the ZNE method [76–78] to mitigate errors in primitive VQCs in which the zero-noise expectation value of an observable is extrapolated from its values at different noise levels. To achieve this, one has to systematically increase the noise in the system and measure the expectation value of the desired observable at different noise strengths. In our case, since the noise is assumed to be only in controlled-not gates, we can increase the noise strength by gate folding [102]: We replace each controlled-not gate with an odd number of controlled-not gates. Since $(U_{\text{CX}}^{(q,q+1)})^2 = I^{(q,q+1)}$, then all odd powers of $U_{\text{CX}}^{(q,q+1)}$ is expected to be the same as one controlled-not gate. However, for the noisy operation ξ_{CX} in Eq. (9), the multiplication of controlled-not gates induces more noise in the system. We perform ZNE for circuits in which every controlled-not gate is replaced with 1, 3, 5, and 7 consecutive gates which approximately correspond to noise strengths of P , $3P$, $5P$, and $7P$, respectively. The error-mitigated result for $P = 0$ is estimated through third-order polynomial extrapolation.

III. ENSEMBLE-LEARNING ALGORITHMS

Ensemble-learning classifiers have been introduced in classical machine-learning literature for enhancing the precision of weak classifiers [103]. In these methods, a group of weak classifiers is combined to make a strong classifier with high accuracy. There are several ensemble learning techniques for classification problems. The most prominent of such algorithms include Bagging [9] and AdaBoost [10,11].

The ZNE method, at best, removes the effect of noise in VQCs, and they usually cannot outperform noise-free quantum computers. Therefore, when error-free shallow circuits are insufficient for accurate classification, the improvement by ZNE is limited. In the following, we adopt two ensemble-learning error mitigation algorithms, namely, Bagging and AdaBoost, for VQCs and show how these methods can enhance our classification accuracy.

A. Ensemble learning: Bagging VQC

The Bagging algorithm has been developed as one of the most successful ensemble-learning techniques in the context of classical machine learning [9]. In the Bagging algorithm, a group of classifiers, each trained independently with different training dataset $\mathcal{D}_i$, are combined to make a stronger one. After training, for any given data, all classifiers assign a class label, and the final prediction is decided by a majority vote among all these results. The simplicity of the Bagging algorithm has made it one of the most popular algorithms in classification problems. Here, we show how a Bagging algorithm can be adapted for VQCs. To implement this, we train L_c different VQCs with shallow circuits, all with equal layers. The difference between these classifiers is in the initialization of the parameters, which results in different optimal values of θ^* . Hence, one gets L_c different VQCs, all trained independently. For unknown data $\mathbf{x}$, we use the majority vote among these L_c classifiers to assign a class label. Therefore the final classifier can be described as

$$\hat{y} = F_{BG}(\mathbf{x}; \theta^*) = \arg \max_k \sum_{l=1}^{L_c} \mathbb{I}(f_l(\mathbf{x}; \theta_l^*) = k), \quad (10)$$

where $f_l(\mathbf{x}; \theta_l^*)$ is a VQC, given in Eq. (8).

To see the performance of the Bagging algorithm, in Fig. 4(a), we plot the test accuracy as a function of L_c for two types of noise-free circuits with $D_l = 2$ and $D_l = 3$ layers, respectively. Each data point is again averaged over 50 random initializations. As expected, in the absence of noise, the performance of the quantum circuit with $D_l = 3$ layers always outperforms the circuit with $D_l = 2$ layers. More importantly, even for such shallow circuits, the accuracy enhances by increasing the number of classifiers L_c such that for $L_c = 10$ one can achieve the accuracy of 0.8856 (for the circuit with $D_l = 2$ layers) and 0.9173 (for the circuit with $D_l = 3$ layers). To achieve a similar accuracy on a single circuit one needs a quantum circuit with $D_l = 6$ layers (0.9205), see Fig. 2. Note that these are all for noise-free computers (i.e., perfect controlled-not gates with $P = 0$) and as we will see later the improvement achieved by Bagging becomes even more pronounced in the presence of noise.

B. Ensemble learning: AdaBoost VQC

AdaBoost is an alternative ensemble-learning algorithm that is used to improve the accuracy of weak classifiers [10,11]. It can be used for those classifiers that slightly outperform a random guess, namely, $0 \leq e \leq (K_c - 1)/K_c$ [11]. While in the Bagging approach, the VQCs are trained in parallel (i.e., independently), in the AdaBoost scheme the VQCs should be trained sequentially. We consider L_c different quantum

classifiers $f_l(\mathbf{x}; \theta_l)$, with $l = 1, 2, \dots, L_c$. In the AdaBoost training process, one assigns a proper weight to each input data $\mathbf{x}_i$ in the loss function. After training each classifier (namely, finding an optimal set of parameters θ^*), the weights are updated for training the next one, based on the performance of the last classifier. Hence, the training procedure for the classifiers is interconnected and can only be accomplished sequentially. For simplicity, we assume that these quantum classifiers have the same circuit design with equal depths. However, each classifier starts with a different random initial parametrization θ_l and uses a different loss function, depending on the weights. The AdaBoost algorithm pursues the following steps to make a single strong classifier via a proper interconnected training method of these L_c classifiers.

(1) Step 1. Initializing the input weights. We assign an initial weight $\mathbf{W}_1 = [w_{1,1}, w_{1,2}, \dots, w_{1,M_{\text{tra}}}]$ with $w_{1,i} = 1/M_{\text{tra}}$ to all the inputs in the dataset.

(2) Step 2. Training the quantum classifier $f_l(\mathbf{x}; \theta_l)$. We train the quantum circuit of the classifier $f_l(\mathbf{x}; \theta_l)$ (initially we start with $l = 1$) using $\mathbf{W}_l$ and the loss function

$$\mathcal{L}_l(\hat{\mathcal{D}}_{\mathbf{W}_l}; \theta_l) = - \sum_{i=1}^{M_{\text{tra}}} w_{l,i} \mathbf{y}_i^T \ln(\mathbf{p}_i(\mathbf{x}_i; \theta_l)), \quad (11)$$

When training finishes one gets the optimal parameter θ_l^* which corresponds to the classifier $f_l(\mathbf{x}; \theta_l^*)$.

(3) Step 3. Computing error rate. For the trained classifier $f_l(\mathbf{x}; \theta_l^*)$ one can compute the error rate as

$$e_l = \sum_{i=1}^{M_{\text{tra}}} w_{l,i} \mathbb{I}(f_l(\mathbf{x}_i; \theta_l^*) \neq y_i). \quad (12)$$

(4) Step 4. Computing the classifier's weight. Based on the error rate e_l , we can assign a weight to the classifier as

$$\alpha_l = \ln \left(\frac{1 - e_l}{e_l} \right) + \ln(K_c - 1). \quad (13)$$

Note that for classifiers better than random guess, namely, $e_l < (K_c - 1)/K_c$, the coefficient α_l is always positive.

(5) Step 5. Updating the input weights. For those input data $\mathbf{x}_i$ that the classifier $f_l(\mathbf{x}_i; \theta_l^*)$ fails to estimate the correct class label y_i , we increase the input weight $w_{l+1,i}$. The reason is that during the training of the next classifier, this input data will have more impact on the loss function and thus might be correctly classified by the next classifier. The input weights are updated as

$$w_{l+1,i} = \frac{w_{l,i}}{Z_l} e^{\alpha_l \mathbb{I}(f_l(\mathbf{x}_i; \theta_l^*) \neq y_i)}, \quad (14)$$

where Z_l is the normalizing factor

$$Z_l = \sum_{i=1}^{M_{\text{tra}}} w_{l,i} e^{\alpha_l \mathbb{I}(f_l(\mathbf{x}_i; \theta_l^*) \neq y_i)}. \quad (15)$$

(6) Step 6. Training the next classifier. Repeat from step 2 until all the L_c classifiers are trained.

(7) Step 7. Combining the classifiers. One can combine the trained classifiers in order to obtain a stronger one. The combination is weighted according to the strength of each

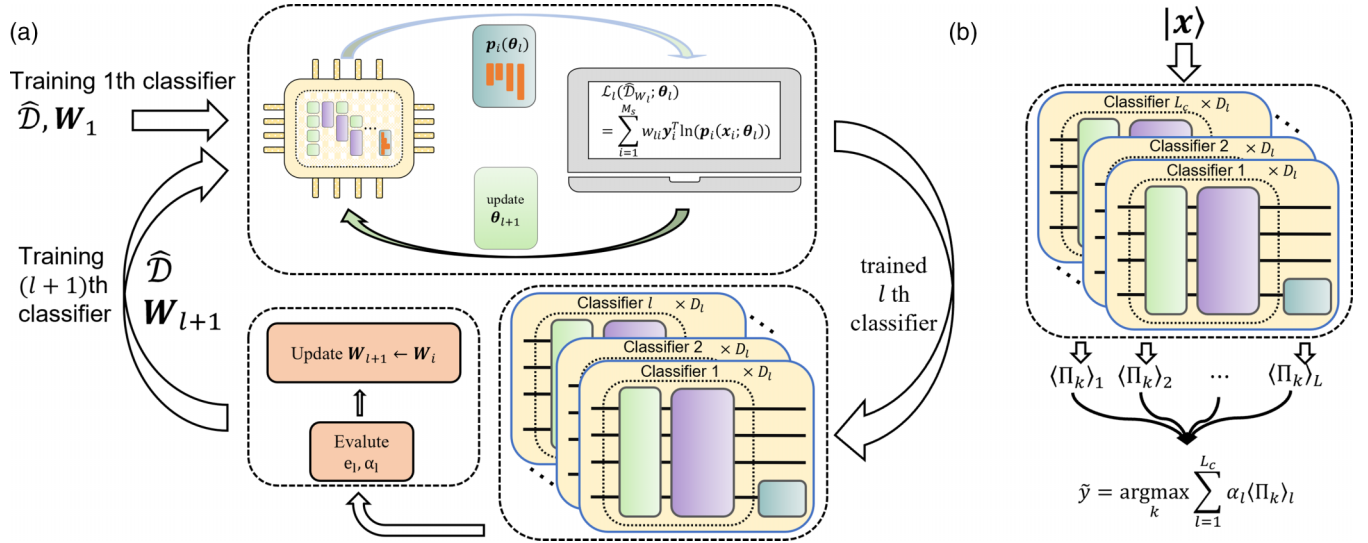


FIG. 3. (a) Training process of AdaBoost VQC. The dataset $\hat{D}$ and the data weights W_l , which is initially taken to be uniform for the first weak VQC, are used to train the l th weak VQC using a shallow circuit, shown in the top dotted box. After training, the error rate e_l is computed with which one can get the classifier's weight α_l . Then the data weights are updated to get W_{l+1} using α_l and the previous data weights W_l . The process repeats until all the L_c classifiers are trained. These trained weak VQCs are combined according to their weights α_l to make a single strong AdaBoost VQC with high accuracy. (b) The unknown input data $|x\rangle$ is fed into L_c different trained weak VQCs. Then the probabilities $\langle \Pi_k \rangle_l$ of measurement outcome k from different weak VQCs are averaged with weights α_l . The final predicted class label is the k with the largest outcome, namely, $\arg \max_k \sum_{l=1}^{L_c} \alpha_l \langle \Pi_k \rangle_l$.

classifier, quantified by α_l :

$$\hat{y} = F_{AB}(x; \theta^*) = \arg \max_k \sum_{l=1}^{L_c} \alpha_l \mathbb{I}(f_l(x; \theta_l^*) = k). \quad (16)$$

The above steps are summarized in Fig. 3(a). Note that the weak performance of the L_c chosen classifiers can be due to different reasons such as shallow circuits or noisy gate operations.

To see the performance of AdaBoost VQC, we first consider shallow VQC circuits whose gate operations are perfect (i.e., the controlled-not gates are noise-free with $P = 0$) and thus their accuracy is only affected by the depth of their circuit. In Fig. 4(b), we plot the test accuracy as a function of L_c for two types of circuits with only $D_l = 2$ and $D_l = 3$ layers. As the figure shows, by increasing the number of classifiers the accuracy increases. In addition, three-layer circuits provide better accuracy in comparison with the two-layer circuits. This is because, in the absence of noise, a three-layer circuit naturally performs better than a two-layer one. One can compare the performance of Bagging and AdaBoost in Figs. 4(a) and 4(b) when the circuit depths are the same. The figures clearly show that AdaBoost can outperform Bagging. For instance, by considering circuits with $D_l = 3$ layers, the AdaBoost VQC with $L_c = 6$ classifiers can achieve an accuracy of 0.95 while the Bagging VQC even with $L_c = 10$ classifiers cannot exceed 0.92 accuracy. This is because, during the AdaBoost sequential training, each classifier is provoked to correct the mistakes of the previous classifier through weight updating. In contrast, in the Bagging algorithm, the classifiers are trained in parallel and independent from each other. Therefore the mistakes are not corrected as efficiently as in the AdaBoost algorithm.

C. Ensemble-learning VQCs on NISQ computers

In this section, we consider noisy quantum computers in which controlled-not gates are noisy and operate according to Eq. (9). The strength of noise is quantified with decoherence rate P , which affects all the two-qubit gates of the circuit equally. We compare four different scenarios: (i) a normal VQC with a deep circuit of $D_l = 12$ layers without ZNE; (ii) a VQC with a deep circuit of $D_l = 12$ layers with ZNE; (iii) Bagging VQC with shallow circuits of $D_l = 2$ and $D_l = 3$ layers with various numbers of classifiers; and (iv) AdaBoost VQC with shallow circuits of $D_l = 2$ and $D_l = 3$ layers with various numbers of classifiers. We first fix the circuit layer $D_l = 2$ for Bagging and AdaBoost and plot the test accuracy as a function of decoherence rate P in Figs. 5(a)–5(c), for $L_c = 3, 6$, and 9 classifiers, respectively. As the figure shows, the test accuracy for a normal VQC with a deep circuit of $D_l = 12$ decays rapidly as P increases. ZNE can indeed enhance the accuracy for such a deep circuit, but for larger P the decay is still significant. Interestingly, a shallow Bagging VQC with $D_l = 2$ can outperform the deep circuit classifier even with ZNE when $P > 0.06$. Increasing the number of classifiers from $L_c = 3$ to 9 slightly improves the performance of Bagging. Remarkably, the AdaBoost algorithm with even shallow circuits of $D_l = 2$ layers can outperform the other scenarios for noise rates of $P > 0.02$ and remains stably high even for very strong decoherence rates up to $P = 0.18$.

Similarly, one can consider the Bagging and the AdaBoost with $D_l = 3$ layers in Figs. 5(d)–5(f) for $L_c = 3, 6$ and 9 classifiers, respectively. In this case, the Bagging and AdaBoost outperform deep circuits with ZNE when $P > 0.02$. As P increases, the performance of Bagging and AdaBoost

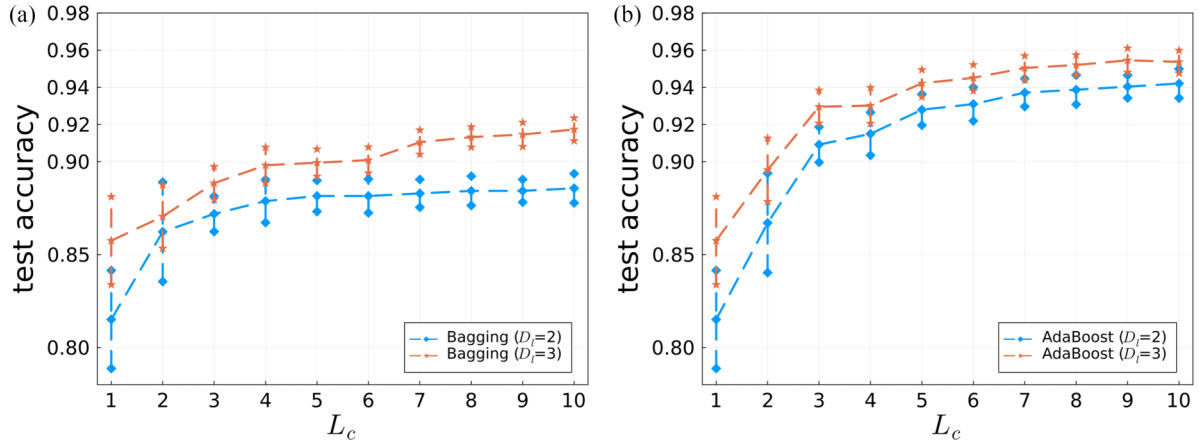


FIG. 4. The performance of ensemble-learning VQCs in noise-free circuits. The test accuracy of MNIST $\{1, 3, 5, 7\}$ classification using ensemble-learning VQCs is plotted as a function of the number of weak classifiers L_c . The ensembles contain noise-free shallow quantum circuits with either $D_l = 2$ or $D_l = 3$ layers. The panels represent: (a) Bagging VQC and (b) AdaBoost VQC. For both algorithms, each data point is averaged over 20 different samples of the initial parameters and the error bars are the standard deviation of those results. Increasing the number of classifiers monotonically enhances the performance of both algorithms. Since the quantum circuits are noise-free, the performance of the circuit with $D_l = 3$ layers is always better than the circuit with $D_l = 2$ layers. It is worth noting that for the same circuit layers D_l and the number of classifiers L_c , the AdaBoost VQC always outperforms the Bagging VQC.

remains fairly close to each other for $L_c = 3$ and $L_c = 6$ classifiers. By increasing the number of classifiers L_c or noise rate P , again AdaBoost outperforms Bagging. Note that

the AdaBoost algorithm is hugely benefited by increasing the number of classifiers due to its interconnected training method, which improves the classifiers based on the mistakes

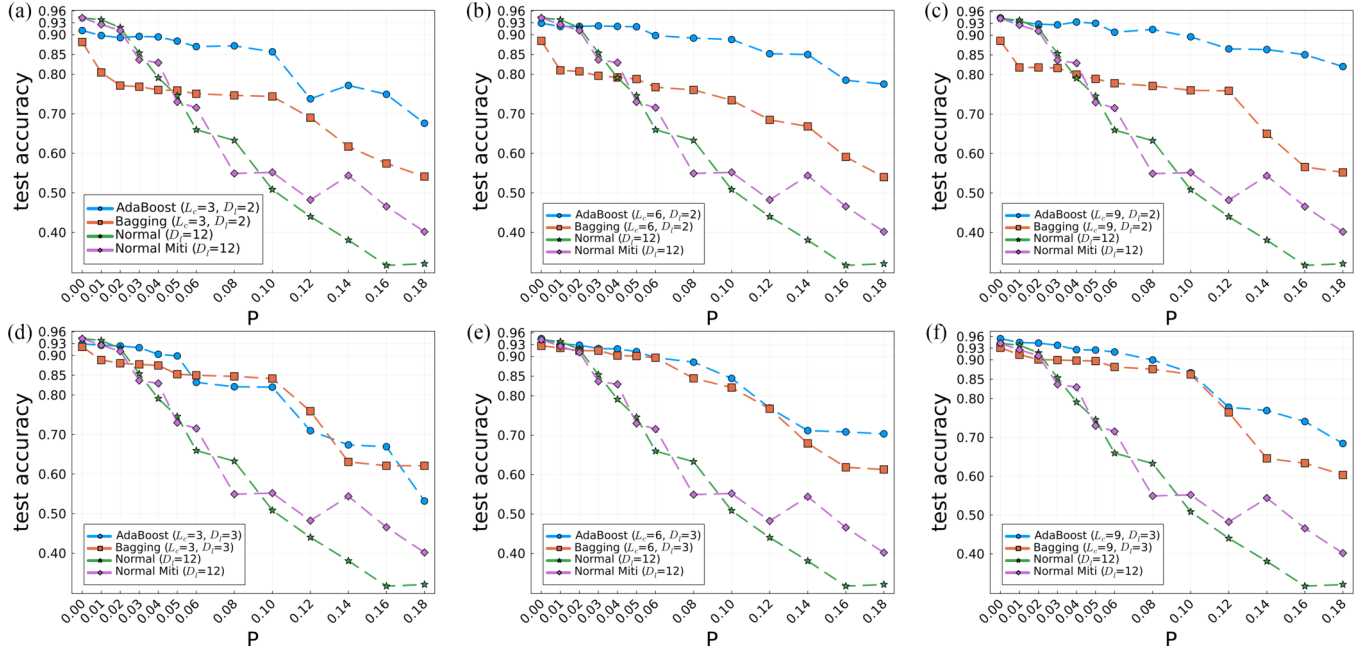


FIG. 5. Comparison of various VQC algorithms on noisy quantum circuits. The test accuracy of four different VQC algorithms is plotted as a function of decoherence rate P . The four strategies include normal VQCs, performed on a deep circuit of $D_l = 12$ layers, with and without ZNE, as well as our ensemble-learning algorithms, namely, Bagging VQC and AdaBoost VQC. In the upper panels, both Bagging and Adaboost are performed on shallow circuits with $D_l = 2$ layers and ensembles of size: (a) $L_c = 3$, (b) 6, and (c) 9 classifiers, respectively. In the lower panels, both Bagging and Adaboost are performed on shallow circuits with $D_l = 3$ layers and ensembles of size: (d) $L_c = 3$, (e) 6, and (f) 9 classifiers, respectively. Each of the data points plotted in these panels is averaged over 50 random samples of initial parameters. The results show that while conventional error mitigation ZNE can indeed enhance the classification accuracy of deep circuits, its performance remains below ensemble-learning classifiers with shallow circuits, as P increases. The best outcome is indeed achieved by AdaBoost whose performance significantly enhances as the number of classifiers increases and remains very robust even at large decoherence rates.

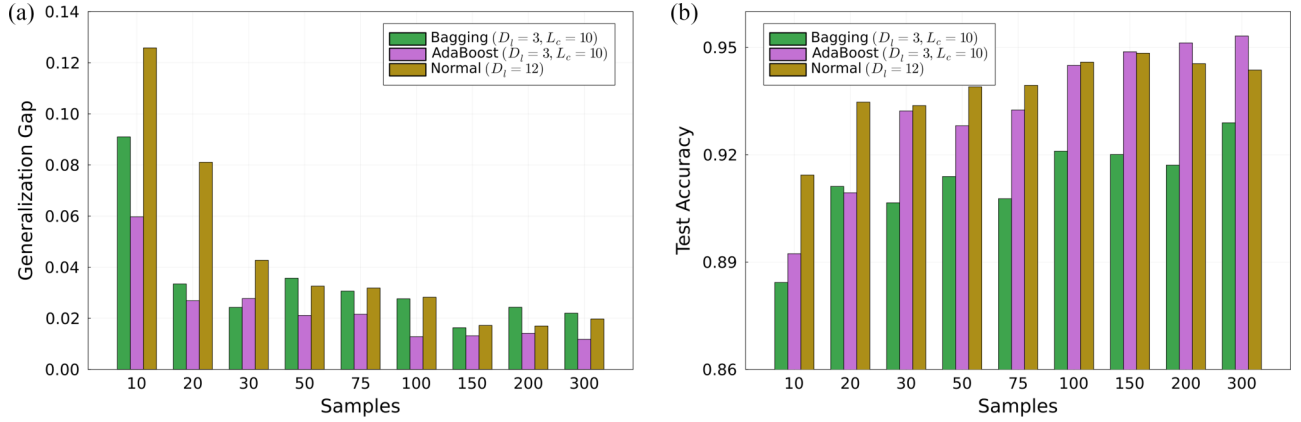


FIG. 6. Comparison of the generalization of various VQC algorithms. (a) The generalization gap and (b) the test accuracy of three VQC algorithms are plotted as various training data samples. The three VQCs include Normal VQC with a single $D_l = 12$ layers circuit, as well as Bagging VQC and AdaBoost VQC with $L_c = 10$ weak classifiers and the depth of each classifier $D_l = 3$ layers. Each of the data bars plotted in these panels is averaged over 10 times with random initial parameters and training data samples but testing the whole test dataset. (a) The result shows that as the training sample size M_{tra} increases, the generalization gap of three VQCs declines. AdaBoost VQC (purple bars) always has the lowest generalization gap than Normal VQC and Bagging VQC. This demonstrates the good generalization ability of AdaBoost VQC. (b) This plot clearly shows that the test accuracy of three VQC increases as the training sample size M_{tra} increases. AdaBoost outperforms normal VQC over the raining sample size of 100. At the same time, AdaBoost VQC has a lower generalization gap than normal VQC. Interestingly, the test accuracy becomes almost a plateau after $M_{\text{tra}} \simeq 100$, where the generalization gap stabilizes.

of the previous ones. Another interesting observation is that in very noisy quantum computers, i.e., large P , AdaBoost with $D_l = 2$ layers is better than AdaBoost with $D_l = 3$ layers. This is because deeper circuits naturally have more two-qubit gates and thus are more susceptible to the effect of noise.

In summary, both our ensemble-learning error mitigation methods, namely, Bagging VQC and AdaBoost VQC, provide a significant improvement over the conventional ZNE method. This is a general behavior and can also be observed for rotation encoding of the input data (results not shown). Moreover, thanks to its interconnected training method, the AdaBoost algorithm can outperform the Bagging, in particular, when the number of classifiers increases. The AdaBoost accuracy enhancement over the other methods becomes even more pronounced when the quantum computer is subjected to strong decoherence, namely, large P .

D. Generalization and trainability of ensemble-learning VQCs on various training samples

Apart from the robustness against decoherence, discussed above, two more issues should be considered in machine learning algorithms, namely, generalization and trainability [47,58,104]. The generalization deals with the performance of the classifier over the unseen test data. The trainability, however, deals with the speed of convergence in order to reach the minimum of the loss function for a given training set. To investigate these issues, without loss of generality, we consider a noise-free environment where all gates are implemented perfectly.

In order to explore the generalization of our protocol, we first define the generalization gap as the difference between

the loss values on the training and test data samples as

$$G(\hat{\mathcal{D}}; \theta^*) = \frac{1}{M_{\text{tra}}} \sum_{i=1}^{M_{\text{tra}}} \mathcal{L}(f(x_i; \theta^*), y_i) - \frac{1}{M_{\text{tes}}} \sum_{i=1}^{M_{\text{tes}}} (\mathcal{L}(f(x'_i; \theta^*), y'_i)), \quad (17)$$

where θ^* are the trained parameters and M_{tes} is the number of test data samples in $\hat{\mathcal{D}}$ and is kept fixed to $M_{\text{tes}} = 726$. Note that the classification function $f(x; \theta^*)$ and the optimal parameter θ^* both depend on the size of the training set M_{tra} . The generalization gap $G(\hat{\mathcal{D}}; \theta^*)$ shows the difference between training and test performance as the size of the training dataset M_{tra} varies. Note that by increasing the size of the training set M_{tra} , one expects that the generalization gap $G(\hat{\mathcal{D}}; \theta^*)$ decreases due to better training. However, this quantity can also detect overfitting. In this case, by increasing M_{tra} the generalization gap initially decreases, showing better training, and then increases, due to overfitting.

To see the behavior of the generalization gap in our protocols, in Fig. 6(a), we plot $G(\hat{\mathcal{D}}; \theta^*)$ as a function of M_{tra} for Bagging, AdaBoost and normal (i.e., no ensemble learning) algorithms. For both the Bagging and the AdaBoost, we use quantum circuits with $D_l = 3$ layers and the total $L_c = 10$ classifiers. On the other hand, for the normal classifier, we rely on a single quantum circuit with $D_l = 12$ layers. For each handwriting digit 1, 3, 5, 7, we randomly chose M_{tra} samples to build up the training dataset. As the figure shows, by increasing the training set M_{tra} the generalization gap shows improvement. While the generalization gap for Bagging and AdaBoost stabilizes after $M_{\text{tra}} \simeq 100$, for the normal algorithm it stabilizes a bit later, after $M_{\text{tra}} \simeq 150$. Notably, the generalization gap reaches lower values for the AdaBoost

algorithm, showing its superiority over other Bagging and normal methods. Interestingly, neither of the algorithms shows overfitting as the training size increases.

The generalization gap alone is not enough to measure the generalization of a classification model over unseen data. For example, a classifier with a small generalization gap and a small test accuracy is not preferred over another classifier with a slightly higher generalization gap but a larger test accuracy. Hence, we consider the test accuracy of three VQCs on various training sample sizes M_{tra} , which explores how many samples are needed to train a classifier with a good performance on the test dataset. We again use Bagging and AdaBoost algorithms with a set of $L_c = 10$ classifiers using circuits with $D_l = 3$ layers. As a reference, we compare these algorithms with a normal classifier made of a single circuit with $D_l = 10$ layers. The test accuracy as a function of training size M_{tra} is shown in Fig. 6(b). As the figure shows, the test accuracy improves as M_{tra} increases. Indeed, the AdaBoost algorithm shows the fastest improvement over the other methods. Interestingly, the test accuracy becomes almost a plateau after $M_{\text{tra}} \simeq 100$, where the generalization gap stabilizes.

Another issue that has to be considered is the trainability of our algorithms. The trainability focuses on the convergence speed in the minimization of the loss function. For the sake of quantification, we consider the average number of iterations I that the convergence of the loss function takes to reach its minimum for a given training size M_{tra} . In order to choose I , we consider the criteria that the loss function should saturate up to a threshold (let's say 10^{-4}) for at least 10 iterations. In other words, $|\mathcal{L}(\hat{\mathcal{D}}; \theta)^{(I)} - \mathcal{L}(\hat{\mathcal{D}}; \theta)^{(I+10)}| \leq 10^{-4}$, which implies that the variation of the loss function remains within 10^{-4} in the next 10 iterations. In Fig. 7, we depict the iteration I as a function of training size M_{tra} for three different algorithms, namely, Bagging, AdaBoost, and normal (i.e., no ensemble learning) method. Each algorithm is repeated 10 times and the obtained iterations are averaged to obtain I . Similar to the previous cases, for Bagging and AdaBoost, we rely on $L_c = 10$ classifiers each with $D_l = 3$ layers. The normal classifier contains a single quantum circuit with $D_l = 12$ layers. While there are some variations in the iterations as M_{tra} increases, the AdaBoost clearly shows the fastest convergence. Interestingly, the optimal training size seems to be $M_{\text{tra}} \simeq 100$, which is consistent with the minimum generalization gap and the saturation of test accuracy.

IV. CLASSIFICATION OF QUANTUM DATA

In this section, we apply our ensemble classification methods to a quantum dataset. The input data are quantum states which are taken from the ground state of a chain of N_q qubits interacting via Hamiltonian

$$H = -J \sum_{i=1}^{N_q-2} \sigma_z^{(i)} \sigma_x^{(i+1)} \sigma_z^{(i+2)} - h_1 \sum_{i=1}^{N_q-1} \sigma_x^{(i)} \sigma_x^{(i+1)} - h_2 \sum_{i=1}^{N_q} \sigma_x^{(i)}, \quad (18)$$

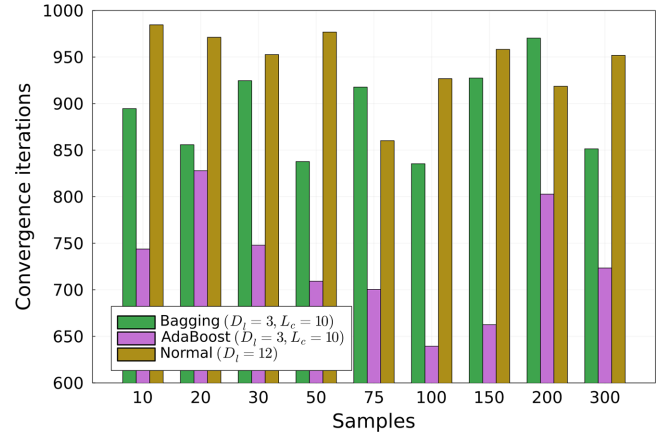


FIG. 7. Comparison of the convergence rate of various VQC algorithms. The convergence iteration of the classifier in three VQC algorithms is plotted as various training data samples. The three VQCs include normal VQC with a deep circuit of $D_l = 12$ layers, as well as Bagging VQC and AdaBoost VQC with weak classifiers $L_c = 10$ and the depth of each classifier is $D_l = 3$ layers. Each of the data bars plotted in this panel is averaged over ten random samples of initial parameters and training data samples. The number of iterations is counted as if the difference of the training loss in the next 10 iterations is less than 10^{-4} . The AdaBoost clearly shows the fastest convergence. In addition, the optimal training size seems to be $M_{\text{tra}} \simeq 100$, which is consistent with the minimum generalization gap and the saturation of test accuracy.

where J is the three-body spin coupling, h_1 is the two-body spin exchange interaction, and h_2 is the magnetic field. Note that the three-body interaction term flips a central spin with the addition of a phase that depends on the quantum states of its neighbors. This Hamiltonian commutes with two string operators

$$X_{\text{odd (even)}} = \prod_{i \in \text{odd (even)}} \sigma_x^{(i)}. \quad (19)$$

This implies that the Hamiltonian has a $\mathbb{Z}_2 \times \mathbb{Z}_2$ symmetry which results in the emergence of a symmetry-protected topological (SPT) phase which is described by a nonlocal order parameter [106,107]. In Ref. [41], the phase diagram of this Hamiltonian has been determined through density matrix renormalization group analysis [105]. The Hamiltonian has three different phases as $(h_1/J, h_2/J)$ vary, namely, antiferromagnetic, paramagnetic, and SPT phases. In the absence of two-body interaction, i.e., $h_1 = 0$, the Hamiltonian becomes solvable via Jordan-Wigner transformation and shows a quantum phase transition from the SPT to the paramagnetic phase at a specific value of h_2/J . Recently, the phase diagram of this system has also been determined through quantum convolution neural networks [41] which has been experimentally realized in superconducting quantum computers for a system of size $N_q = 7$ [67].

Here, we use our ensemble classification methods to determine the phase diagram of the system. The quantum circuit is exactly the same as before, shown in Fig. 1(a), with one important difference. Since the input is itself a quantum state, the encoder is no longer needed and the quantum state can directly be fed into the parameterized circuit. Similar to the

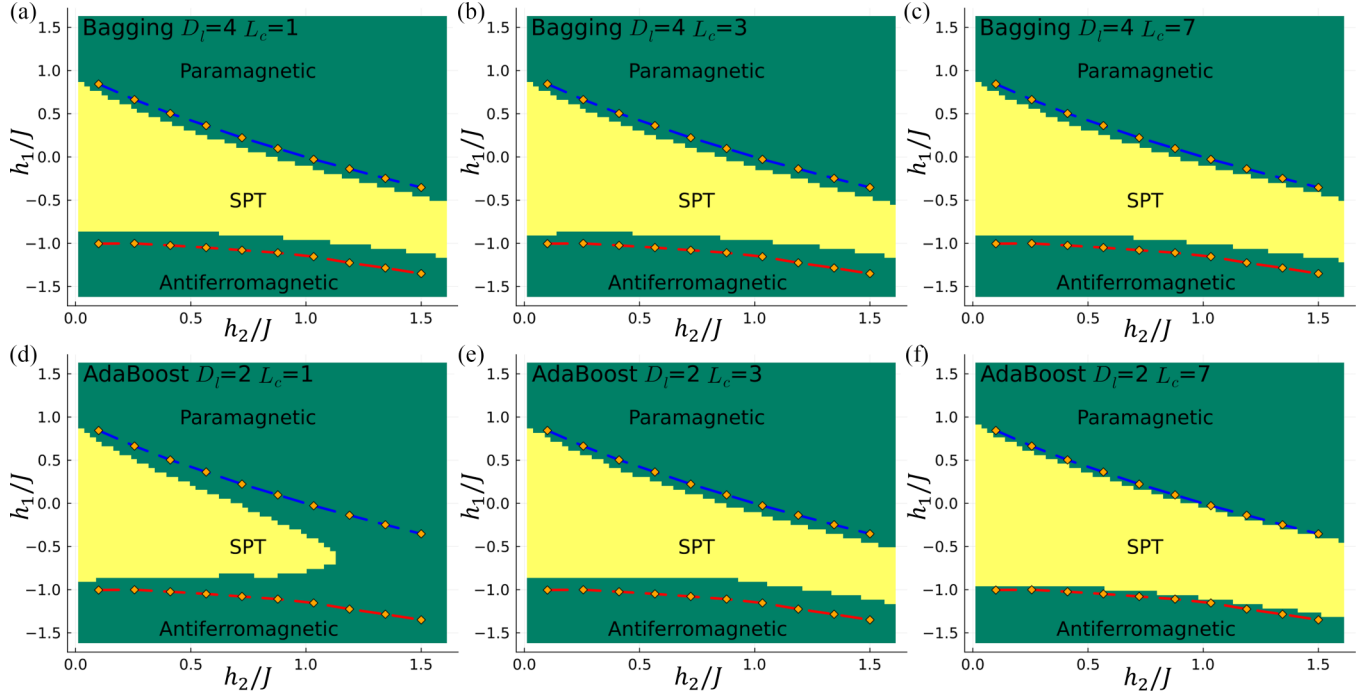


FIG. 8. VQC for quantum datasets. The test performance for phase recognition of the ground state of the SPT Hamiltonian (18) with 15 qubits as a function of h_1/J and h_2/J . The upper panels show the performance of Bagging for quantum circuits with $D_l = 4$ layers with ensembles of size: (a) $L_c = 1$, (b) 3, and (c) 7 classifiers, respectively. The lower panels show the performance of AdaBoost for quantum circuits with only $D_l = 2$ layers with ensembles of size: (d) $L_c = 1$, (e) 3, and (f) 7 classifiers, respectively. The blue and red lines represent the real phase boundaries computed through density matrix renormalization group [41,105]. Note that using a single circuit $L_c = 1$ is not really an ensemble learning but we just include it to show how the results improve as the number of classifiers increases.

approach of Ref. [41], we only measure the last qubit despite having three phases, i.e., three classes. This method labels the phases as SPT and non-SPT phases. Since antiferromagnetic and paramagnetic phases are well separated and have no boundary they can be easily recognized in the phase diagram, as we will see in the following.

First, we focus on the Bagging algorithm for phase recognition of the SPT Hamiltonian with $N_q = 15$ qubits using an ensemble of circuits with $D_l = 4$ layers. We consider the phase diagram in the $(h_1/J, h_2/J)$ plane with the resolution of 64×64 pixels. For training the circuit, We randomly select the ground states of $M_{\text{tra}} = 400$ random samples in the $(h_1/J, h_2/J)$ plane, as our training data. In Figs. 8(a)–8(c), we plot the result of our Bagging VQC for an ensemble of $L_c = 1, 3$, and 7 classifiers, respectively. The phase boundaries, computed by density matrix renormalization group [41,105], are plotted by the blue and red lines. As evident in the figures, Bagging VQC can indeed capture the phase diagram and the precision becomes better as the number of classifiers increases. It is worth emphasizing that for shallower circuits with the depth $D_l < 4$ layers, the precision for capturing the phase diagram goes down (results not shown). In particular, the performance is poor for circuits with $D_l = 2$ layers, no matter how many classifiers we use. This shows that increasing the number of classifiers alone cannot compensate the circuit depth. This is because the classifiers are trained independently and their weakness cannot be improved during training.

Second, we also exploit AdaBoost for capturing the phase diagram of the Hamiltonian with very shallow circuits of $D_l = 2$ layers. Similar to the previous cases, we use the same circuit as shown in Fig. 1(a) without the encoder part. We use the same dataset that we used for the Bagging algorithm. In Figs. 8(d)–8(f), we depict the phase diagram of the system using AdaBoost circuits with the depth of $D_l = 2$ layers and $L_c = 1, 3$, and 7 classifiers, respectively. Note that the AdaBoost can only become effective for more than one classifier. As the figures clearly show, the AdaBoost protocol can indeed determine the phase diagram even with shallow circuits with only $D_l = 2$ layers. As expected, the precision is improved as the number of classifiers L_c increases. In particular, for $L_c = 7$, the phase boundaries between the SPT and the other phases are captured quite precisely. The fact that circuits with only $D_l = 2$ layers are enough for recognizing the phase boundaries already shows the superiority of AdaBoost VQC over Bagging VQC. As mentioned before, this is because in AdaBoost VQC the training of classifiers is not independent of each other in such a way that each classifier tries to correct the errors of the previous ones through weight updating.

V. CONCLUSIONS

We introduced two ensemble-learning error mitigation algorithms, namely, Bagging and AdaBoost, for VQCs. These algorithms can significantly enhance the precision of classification using only shallow quantum circuits with very

few parameters to train. Our protocols have been tested on both classical (handwriting digits) and quantum (the phase recognition of an SPT Hamiltonian) datasets. Considering imperfect NISQ computers, our ensemble-learning error mitigation methods significantly outperform the ZNE method in classification tasks. Thanks to its interconnecting training approach, which tends to correct the mistakes of one classifier in the training of the next one, the AdaBoost method achieves better accuracy and shows better robustness against noise than the Bagging algorithm. The superiority of AdaBoost over ZNE and Bagging becomes even more prominent when the number of classifiers increases, in particular at the large noise limit. In addition, we demonstrate the generalization and trainability power of our protocols with respect to the training size. Indeed, AdaBoost outperforms Bagging and normal VQCs for both generalization and trainability.

Our ensemble-learning error mitigation techniques are very general. For classification problems, they are applicable to both classical and quantum datasets and work for both amplitude and rotation encodings. The application of our ensemble-learning error mitigation methods is not limited to classification and can be generalized to other supervised machine-learning problems such as Kernel learning and regression. While in our paper we rely on hardware efficient ansatz, the protocols are applicable to any circuit design. Moreover, it can also be used for nonvariational classification methods such as quantum support vector machines. Our ensemble-learning VQCs are NISQ-friendly and very distinct from other ensemble-learning proposals [82–86] which are hardware demanding, relying on multi-qubit controlled unitaries and quantum subroutines such as quantum phase estimation, Grover search and quantum mean estimation. Therefore our approach can solve a broad category of machine learning problems with today’s NISQ technologies.

Recently, it has been shown that the measurement outcomes of noisy shallow quantum circuits on Pauli observables may be simulable on classical computers [108–110]. This, however, does not mean that Bagging and AdaBoost algorithms can be simulated classically. The reasons are twofold. First, these algorithms can also be applied to quantum datasets, which in general cannot be handled by classical simulation. Second, both AdaBoost and Bagging can

also be applied on noiseless quantum circuits, which cannot be simulated classically, and enhance the accuracy of such classifiers. Therefore both Bagging and AdaBoost can achieve better performance beyond the capability of classical simulation.

All computer codes used in this paper are available in Ref. [111].

ACKNOWLEDGMENTS

The authors gratefully thank the National Natural Science Foundation of China (Grants No. 12050410253, No. 92065115, No. 12274059, No. 92265208, No. 12225507 and No. 12088101), the Ministry of Science and Technology of China (Grant No. QNJ2021167001L), and the National Key R&D Program of China (Grant No. 2018YFA0306703) for their support. They also thank Guanyu Zhou for helpful discussions and Chu Guo for the software “VQC.jl.”

APPENDIX: TRAINING THE QUANTUM CIRCUIT

For numerical simulations, we rely on JULIA packages “VQC.jl” and “QuantumCircuit.jl”. In the training process of our VQCs, we use Adam optimizer [99], which is a gradient-based method, with a learning rate of 5×10^{-3} , to update the quantum circuit parameters θ . The gradients are obtained by Automatic differentiation methods supported by the VQC.jl package. The optimization iterations of the training procedure are 500 times for the weak quantum classifiers in both AdaBoost VQC and Bagging VQC and 1500 times for the normal deep VQC. In order to be initialization-independent, for noise-free and noisy circuits, the performance is averaged over 50 and 20 random initial samples, respectively.

For quantum classification of the SPT Hamiltonian, the training dataset takes the ground state of $M_{\text{tra}} = 400$ random samples in the $(h_1/J, h_2/J)$ plane. To show the performance of the classifier, we depict the phase diagram with the resolution of 64×64 averaged in the $(h_1/J, h_2/J)$ plane, as shown in Fig. 8. The optimization iteration is fixed to 1000 and each data point has been averaged over ten different random initial samples.

-
- [1] T. M. Mitchell, *Machine Learning* (McGraw-Hill, New York, 1997).
 - [2] S. B. Kotsiantis, Supervised machine learning: A review of classification techniques, in *Proceedings of the 2007 Conference on Emerging Artificial Intelligence Applications in Computer Engineering: Real World AI Systems with Applications in EHealth, HCI, Information Retrieval and Pervasive Technologies* (IOS Press, NLD, 2007), pp. 3–24.
 - [3] M.-L. Zhang and Z.-H. Zhou, A review on multi-label learning algorithms, *IEEE Trans. Knowl. Data Eng.* **26**, 1819 (2014).
 - [4] J. Tolles and W. J. Meurer, Logistic regression: Relating patient characteristics to outcomes, *JAMA* **316**, 533 (2016).
 - [5] J. R. Quinlan, Induction of decision trees, *Mach. Learn.* **1**, 81 (1986).
 - [6] T. Cover and P. Hart, Nearest neighbor pattern classification, *IEEE Trans. Inf. Theory* **13**, 21 (1967).
 - [7] C. Cortes and V. Vapnik, Support-vector networks, *Mach. Learn.* **20**, 273 (1995).
 - [8] G. Zhang, Neural networks for classification: a survey, *IEEE Trans. Syst. Man Cybern. Syst.* **30**, 451 (2000).
 - [9] L. Breiman, Bagging predictors, *Mach. Learn.* **24**, 123 (1996).
 - [10] Y. Freund and R. E. Schapire, A decision-theoretic generalization of on-line learning and an application to boosting, in *Computational Learning Theory*, edited by P. Vitányi (Springer Berlin Heidelberg, Berlin, Heidelberg, 1995), pp. 23–37.
 - [11] T. Hastie, S. Rosset, J. Zhu, and H. Zou, Multi-class AdaBoost, *Stat Interface* **2**, 349 (2009).

- [12] R. Barends, A. Shabani, L. Lamata, J. Kelly, A. Mezzacapo, U. L. Heras, R. Babbush, A. G. Fowler, B. Campbell, Y. Chen, Z. Chen, B. Chiaro, A. Dunsworth, E. Jeffrey, E. Lucero, A. Megrant, J. Y. Mutus, M. Neeley, C. Neill, P. J. J. O'Malley *et al.*, Digitized adiabatic quantum computing with a superconducting circuit, *Nature (London)* **534**, 222 (2016).
- [13] T. Hensgens, T. Fujita, L. Janssen, X. Li, C. J. Van Diepen, C. Reichl, W. Wegscheider, S. Das Sarma, and L. M. K. Vandersypen, Quantum simulation of a Fermi-Hubbard model using a semiconductor quantum dot array, *Nature (London)* **548**, 70 (2017).
- [14] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, Quantum supremacy using a programmable superconducting processor, *Nature (London)* **574**, 505 (2019).
- [15] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, Hartree-fock on a superconducting qubit quantum computer, *Science* **369**, 1084 (2020).
- [16] Y. Wu, W.-S. Bao, S. Cao, F. Chen, M.-C. Chen, X. Chen, T.-H. Chung, H. Deng, Y. Du, D. Fan *et al.*, Strong quantum computational advantage using a superconducting quantum processor, *Phys. Rev. Lett.* **127**, 180501 (2021).
- [17] S. Bravyi, O. Dial, J. M. Gambetta, D. Gil, and Z. Nazario, The future of quantum computing with superconducting qubits, *J. Appl. Phys.* **132**, 160902 (2022).
- [18] C. Hempel, C. Maier, J. Romero, J. McClean, T. Monz, H. Shen, P. Jurcevic, B. P. Lanyon, P. Love, R. Babbush, A. Aspuru-Guzik, R. Blatt, and C. F. Roos, Quantum chemistry calculations on a trapped-ion quantum simulator, *Phys. Rev. X* **8**, 031022 (2018).
- [19] C. Kokail, C. Maier, R. van Bijnen, T. Brydges, M. K. Joshi, P. Jurcevic, C. A. Muschik, P. Silvi, R. Blatt, C. F. Roos *et al.*, Self-verifying variational quantum simulation of lattice models, *Nature (London)* **569**, 355 (2019).
- [20] M. Ringbauer, M. Meth, L. Postler, R. Stricker, R. Blatt, P. Schindler, and T. Monz, A universal qudit quantum processor with trapped ions, *Nat. Phys.* **18**, 1053 (2022).
- [21] C. Noel, P. Niroula, D. Zhu, A. Risinger, L. Egan, D. Biswas, M. Cetina, A. V. Gorshkov, M. J. Gullans, D. A. Huse *et al.*, Measurement-induced quantum phases realized in a trapped-ion quantum computer, *Nat. Phys.* **18**, 760 (2022).
- [22] C. Monroe, W. C. Campbell, L.-M. Duan, Z.-X. Gong, A. V. Gorshkov, P. W. Hess, R. Islam, K. Kim, N. M. Linke, G. Pagano, P. Richerme, C. Senko, and N. Y. Yao, Programmable quantum simulations of spin systems with trapped ions, *Rev. Mod. Phys.* **93**, 025001 (2021).
- [23] M. Schreiber, S. S. Hodgman, P. Bordia, H. P. Lüschen, M. H. Fischer, R. Vosk, E. Altman, U. Schneider, and I. Bloch, Observation of many-body localization of interacting fermions in a quasirandom optical lattice, *Science* **349**, 842 (2015).
- [24] C. Gross and I. Bloch, Quantum simulations with ultracold atoms in optical lattices, *Science* **357**, 995 (2017).
- [25] P. Sompet, S. Hirthe, D. Bourgund, T. Chalopin, J. Bibo, J. Koepsell, P. Bojović, R. Verresen, F. Pollmann, G. Salomon *et al.*, Realizing the symmetry-protected Haldane phase in Fermi-Hubbard ladders, *Nature (London)* **606**, 484 (2022).
- [26] M. Saffman, Quantum computing with atomic qubits and Rydberg interactions: progress and challenges, *J. Phys. B* **49**, 202001 (2016).
- [27] A. Aspuru-Guzik and P. Walther, Photonic quantum simulators, *Nat. Phys.* **8**, 285 (2012).
- [28] H.-S. Zhong, H. Wang, Y.-H. Deng, M.-C. Chen, L.-C. Peng, Y.-H. Luo, J. Qin, D. Wu, X. Ding, Y. Hu, P. Hu, X.-Y. Yang, W.-J. Zhang, H. Li, Y. Li, X. Jiang, L. Gan, G. Yang, L. You, Z. Wang *et al.*, Quantum computational advantage using photons, *Science* **370**, 1460 (2020).
- [29] T. Dai, Y. Ao, J. Bao, J. Mao, Y. Chi, Z. Fu, Y. You, X. Chen, C. Zhai, B. Tang *et al.*, Topologically protected quantum entanglement emitters, *Nat. Photon.* **16**, 248 (2022).
- [30] A. J. Daley, I. Bloch, C. Kokail, S. Flannigan, N. Pearson, M. Troyer, and P. Zoller, Practical quantum advantage in quantum simulation, *Nature (London)* **607**, 667 (2022).
- [31] M. Schuld, M. Fingerhuth, and F. Petruccione, Implementing a distance-based classifier with a quantum interference circuit, *Europhys. Lett.* **119**, 60002 (2017).
- [32] P. Rebentrost, M. Mohseni, and S. Lloyd, Quantum support vector machine for big data classification, *Phys. Rev. Lett.* **113**, 130503 (2014).
- [33] Z. Li, X. Liu, N. Xu, and J. Du, Experimental realization of a quantum support vector machine, *Phys. Rev. Lett.* **114**, 140504 (2015).
- [34] S. Lloyd, M. Mohseni, and P. Rebentrost, Quantum algorithms for supervised and unsupervised machine learning, *arXiv:1307.0411*.
- [35] N. Wiebe, A. Kapoor, and K. M. Svore, Quantum algorithms for nearest-neighbor methods for supervised and unsupervised learning, *Quantum Inf. Comput.* **15**, 316 (2015).
- [36] S. Lu and S. L. Braunstein, Quantum decision tree classifier, *Quantum Inf. Process.* **13**, 757 (2014).
- [37] E. Farhi and S. Gutmann, Quantum computation and decision trees, *Phys. Rev. A* **58**, 915 (1998).
- [38] J. Romero, J. P. Olson, and A. Aspuru-Guzik, Quantum autoencoders for efficient compression of quantum data, *Quantum Sci. Technol.* **2**, 045001 (2017).
- [39] E. Grant, M. Benedetti, S. Cao, A. Hallam, J. Lockhart, V. Stojevic, A. G. Green, and S. Severini, Hierarchical quantum classifiers, *npj Quantum Inf.* **4**, 65 (2018).
- [40] E. Farhi and H. Neven, Classification with quantum neural networks on near term processors, *arXiv:1802.06002*.
- [41] I. Cong, S. Choi, and M. D. Lukin, Quantum convolutional neural networks, *Nat. Phys.* **15**, 1273 (2019).
- [42] N. Killoran, T. R. Bromley, J. M. Arrazola, M. Schuld, N. Quesada, and S. Lloyd, Continuous-variable quantum neural networks, *Phys. Rev. Res.* **1**, 033063 (2019).
- [43] C. Zoufal, A. Lucchi, and S. Woerner, Quantum generative adversarial networks for learning and loading random distributions, *npj Quantum Inf.* **5**, 103 (2019).
- [44] M. Schuld, A. Bocharov, K. M. Svore, and N. Wiebe, Circuit-centric quantum classifiers, *Phys. Rev. A* **101**, 032308 (2020).
- [45] J. Liu, K. H. Lim, K. L. Wood, W. Huang, C. Guo, and H.-L. Huang, Hybrid quantum-classical convolutional neural networks, *Sci. China Phys. Mech. Astron.* **64**, 290311 (2021).
- [46] X. Hou, G. Zhou, Q. Li, S. Jin, and X. Wang, A duplication-free quantum neural network for universal approximation, *Sci. China Phys. Mech. Astron.* **66**, 270362 (2023).
- [47] Y. Du, Y. Yang, D. Tao, and M.-H. Hsieh, Demystify problem-dependent power of quantum neural networks on multi-class classification, *Phys. Rev. Lett.* **131**, 140601 (2023).

- [48] K. Bharti, A. Cervera-Lierta, T. H. Kyaw, T. Haug, S. Alperin-Lea, A. Anand, M. Degroote, H. Heimonen, J. S. Kottmann, T. Menke, W.-K. Mok, S. Sim, L.-C. Kwek, and A. Aspuru-Guzik, Noisy intermediate-scale quantum algorithms, *Rev. Mod. Phys.* **94**, 015004 (2022).
- [49] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, Variational quantum algorithms, *Nat. Rev. Phys.* **3**, 625 (2021).
- [50] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, A variational eigenvalue solver on a photonic quantum processor, *Nat. Commun.* **5**, 4213 (2014).
- [51] A. Kandala, A. Mezzacapo, K. Temme, M. Takita, M. Brink, J. M. Chow, and J. M. Gambetta, Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets, *Nature (London)* **549**, 242 (2017).
- [52] O. Higgott, D. Wang, and S. Brierley, Variational quantum computation of excited states, *Quantum* **3**, 156 (2019).
- [53] D. Wang, O. Higgott, and S. Brierley, Accelerated variational quantum eigensolver, *Phys. Rev. Lett.* **122**, 140504 (2019).
- [54] C. Lyu, V. Montenegro, and A. Bayat, Accelerated variational algorithms for digital quantum simulation of many-body ground states, *Quantum* **4**, 324 (2020).
- [55] C. Lyu, X. Xu, M.-H. Yung, and A. Bayat, Symmetry enhanced variational quantum spin eigensolver, *Quantum* **7**, 899 (2023).
- [56] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, Quantum machine learning, *Nature (London)* **549**, 195 (2017).
- [57] K. Mitarai, M. Negoro, M. Kitagawa, and K. Fujii, Quantum circuit learning, *Phys. Rev. A* **98**, 032309 (2018).
- [58] Y. Qian, X. Wang, Y. Du, X. Wu, and D. Tao, The dilemma of quantum neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* **1** (2022).
- [59] S. Lu, L.-M. Duan, and D.-L. Deng, Quantum adversarial machine learning, *Phys. Rev. Res.* **2**, 033212 (2020).
- [60] Y. Du, M.-H. Hsieh, T. Liu, D. Tao, and N. Liu, Quantum noise protects quantum classifiers against adversaries, *Phys. Rev. Res.* **3**, 023153 (2021).
- [61] N. Liu and P. Wittek, Vulnerability of quantum classification to adversarial perturbations, *Phys. Rev. A* **101**, 062331 (2020).
- [62] W. Ren, W. Li, S. Xu, K. Wang, W. Jiang, F. Jin, X. Zhu, J. Chen, Z. Song, P. Zhang *et al.*, Experimental quantum adversarial learning with programmable superconducting qubits, *Nat. Comput. Sci.* **2**, 711 (2022).
- [63] E. Farhi, J. Goldstone, and S. Gutmann, A quantum approximate optimization algorithm, [arXiv:1411.4028](https://arxiv.org/abs/1411.4028).
- [64] C. Bravo-Prieto, R. LaRose, M. Cerezo, Y. Subasi, L. Cincio, and P. J. Coles, Variational quantum linear solver, *Quantum* **7**, 1188 (2023).
- [65] X. Xu, J. Sun, S. Endo, Y. Li, S. C. Benjamin, and X. Yuan, Variational algorithms for linear algebra, *Sci. Bull.* **66**, 2181 (2021).
- [66] H.-Y. Huang, K. Bharti, and P. Rebentrost, Near-term quantum algorithms for linear systems of equations with regression loss functions, *New J. Phys.* **23**, 113021 (2021).
- [67] J. L. Beckey, M. Cerezo, A. Sone, and P. J. Coles, Variational quantum algorithm for estimating the quantum Fisher information, *Phys. Rev. Res.* **4**, 013083 (2022).
- [68] R. Kaubruegger, P. Silvi, C. Kokail, R. van Bijnen, A. M. Rey, J. Ye, A. M. Kaufman, and P. Zoller, Variational spin-squeezing algorithms on programmable quantum sensors, *Phys. Rev. Lett.* **123**, 260505 (2019).
- [69] J. J. Meyer, J. Borregaard, and J. Eisert, A variational toolbox for quantum multi-parameter estimation, *npj Quantum Inf.* **7**, 89 (2021).
- [70] L. Banchi, Robust quantum classifiers via NISQ adversarial learning, *Nat. Comput. Sci.* **2**, 699 (2022).
- [71] D.-L. Deng, Quantum enhanced convolutional neural networks for NISQ computers, *Sci. China Phys. Mech. Astron.* **64**, 100331 (2021).
- [72] W. Gong and D.-L. Deng, Universal adversarial examples and perturbations for quantum classifiers, *Natl. Sci. Rev.* **9**, nwab130 (2021).
- [73] A. V. Uvarov, A. S. Kardashin, and J. D. Biamonte, Machine learning phase transitions with a quantum processor, *Phys. Rev. A* **102**, 012415 (2020).
- [74] J. Herrmann, S. M. Llima, A. Remm, P. Zapletal, N. A. McMahon, C. Scarato, F. Swiadek, C. K. Andersen, C. Hellings, S. Krinner, N. Lacroix, S. Lazar, M. Kerschbaum, D. C. Zanuz, G. J. Norris, M. J. Hartmann, A. Wallraff, and C. Eichler, Realizing quantum convolutional neural networks on a superconducting quantum processor to recognize quantum phases, *Nat. Commun.* **13**, 4144 (2022).
- [75] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, Supervised learning with quantum-enhanced feature spaces, *Nature (London)* **567**, 209 (2019).
- [76] Y. Li and S. C. Benjamin, Efficient variational quantum simulator incorporating active error minimization, *Phys. Rev. X* **7**, 021050 (2017).
- [77] K. Temme, S. Bravyi, and J. M. Gambetta, Error mitigation for short-depth quantum circuits, *Phys. Rev. Lett.* **119**, 180509 (2017).
- [78] A. Kandala, K. Temme, A. D. Córcoles, A. Mezzacapo, J. M. Chow, and J. M. Gambetta, Error mitigation extends the computational reach of a noisy quantum processor, *Nature (London)* **567**, 491 (2019).
- [79] S. Endo, Z. Cai, S. C. Benjamin, and X. Yuan, Hybrid quantum-classical algorithms and quantum error mitigation, *J. Phys. Soc. Jpn.* **90**, 032001 (2021).
- [80] D. Qin, X. Xu, and Y. Li, An overview of quantum error mitigation formulas, *Chin. Phys. B* **31**, 090306 (2022).
- [81] Z. Cai, R. Babbush, S. C. Benjamin, S. Endo, W. J. Huggins, Y. Li, J. R. McClean, and T. E. O'Brien, Quantum error mitigation, [arXiv:2210.00921](https://arxiv.org/abs/2210.00921).
- [82] M. Schuld and F. Petruccione, Quantum ensembles of quantum classifiers, *Sci. Rep.* **8**, 2772 (2018).
- [83] X. Wang, Y. Ma, M.-H. Hsieh, and M.-H. Yung, Quantum speedup in adaptive boosting of binary classification, *Sci. China Phys. Mech. Astron.* **64**, 220311 (2021).
- [84] S. Arunachalam and R. Maity, Quantum Boosting, in *Proceedings of the 37th International Conference on Machine Learning* (PMLR, 2020), pp. 377–387.
- [85] A. Izdebski and R. de Wolf, Improved quantum boosting, in *31st Annual European Symposium on Algorithms (ESA 2023)*, edited by I. L. Górtz, M. Farach-Colton, S. J. Puglisi, G. Herman, Leibniz International Proceedings in Informatics

- (LIPIcs) Vol. 274 (Schloss Dagstuhl-Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 2023), pp. 641–6416.
- [86] A. Macaluso, L. Clissa, S. Lodi, and C. Sartori, Quantum ensemble for classification, [arXiv:2007.01028](#).
 - [87] A. Y. Kitaev, Quantum measurements and the Abelian stabilizer problem, [arXiv:quant-ph/9511026](#).
 - [88] A. Nayak and F. Wu, The quantum query complexity of approximating the median and related statistics, in *Proceedings of the Thirty-First Annual ACM Symposium on Theory of Computing*, STOC '99 (Association for Computing Machinery, New York, NY, 1999), pp. 384–393.
 - [89] G. Brassard, F. Dupuis, S. Gambs, and A. Tapp, An optimal quantum algorithm to approximate the mean and its application for approximating the median of a set of points over an arbitrary distance, [arXiv:1106.4267](#).
 - [90] L. K. Grover, A fast quantum mechanical algorithm for database search, in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '96 (Association for Computing Machinery, New York, NY, 1996), pp. 212–219.
 - [91] R. Qin, Z. Liang, J. Cheng, P. Kogge, and Y. Shi, Improving quantum classifier performance in nisy computers by voting strategy from ensemble learning, [arXiv:2210.01656](#).
 - [92] M. T. West, M. Sevier, and M. Usman, Reflection equivariant quantum neural networks for enhanced image classification, *Mach. Learn.: Sci. Technol.* **4**, 035027 (2023).
 - [93] P. A. M. Casares, Circuit implementation of bucket brigade gram for quantum state preparation, [arXiv:2006.11761](#).
 - [94] N. Jiang, Y.-F. Pu, W. Chang, C. Li, S. Zhang, and L.-M. Duan, Experimental realization of 105-qubit random access quantum memory, *npj Quantum Inf.* **5**, 28 (2019).
 - [95] D. K. Park, F. Petruccione, and J.-K. K. Rhee, Circuit-based quantum random access memory for classical data, *Sci. Rep.* **9**, 3949 (2019).
 - [96] V. Giovannetti, S. Lloyd, and L. Maccone, Quantum random access memory, *Phys. Rev. Lett.* **100**, 160501 (2008).
 - [97] M. Schuld, R. Sweke, and J. J. Meyer, Effect of data encoding on the expressive power of variational quantum-machine-learning models, *Phys. Rev. A* **103**, 032430 (2021).
 - [98] R. LaRose and B. Coyle, Robust data encodings for quantum classifiers, *Phys. Rev. A* **102**, 032420 (2020).
 - [99] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization, [arXiv:1412.6980](#).
 - [100] E. Alpaydin and C. Kaynak, Optical recognition of handwritten digits, UCI Machine Learning Repository (1998), doi:<https://doi.org/10.24432/C50P49>.
 - [101] F. Bao, H. Deng, D. Ding, R. Gao, X. Gao, C. Huang, X. Jiang, H.-S. Ku, Z. Li, X. Ma, X. Ni, J. Qin, Z. Song, H. Sun, C. Tang, T. Wang, F. Wu, T. Xia, W. Yu, F. Zhang *et al.*, Fluxonium: An alternative qubit platform for high-fidelity operations, *Phys. Rev. Lett.* **129**, 010502 (2022).
 - [102] T. Giurgica-Tiron, Y. Hindy, R. LaRose, A. Mari, and W. J. Zeng, Digital zero noise extrapolation for quantum error mitigation, in *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)* (IEEE, Denver, USA, 2020), pp. 306–316.
 - [103] O. Sagi and L. Rokach, Ensemble learning: A survey, *WIREs Data Min. Knowl.* **8**, e1249 (2018).
 - [104] E. Gil-Fuster, J. Eisert, and C. Bravo-Prieto, Understanding quantum machine learning also requires rethinking generalization, [arXiv:2306.13461](#).
 - [105] I. P. McCulloch, Infinite size density matrix renormalization group, revisited, [arXiv:0804.2509](#).
 - [106] F. Pollmann and A. M. Turner, Detection of symmetry-protected topological phases in one dimension, *Phys. Rev. B* **86**, 125441 (2012).
 - [107] J. Haegeman, D. Pérez-García, I. Cirac, and N. Schuch, Order parameter for symmetry-protected phases in one dimension, *Phys. Rev. Lett.* **109**, 050402 (2012).
 - [108] N. A. Nemkov, E. O. Kiktenko, and A. K. Fedorov, Fourier expansion in variational quantum algorithms, *Phys. Rev. A* **108**, 032406 (2023).
 - [109] E. Fontana, M. S. Rudolph, R. Duncan, I. Rungger, and C. Cîrstoiu, Classical simulations of noisy variational quantum circuits, [arXiv:2306.05400](#).
 - [110] Y. Shao, F. Wei, S. Cheng, and Z. Liu, Simulating quantum mean values in noisy variational quantum algorithms: A polynomial-scale approach, [arXiv:2306.05804](#).
 - [111] <https://github.com/LeeQY1996/Ensemble-learning-variational-shallow-circuit-quantum-classifiers>.